

C# Self-Study Report

C# Fundamentals - Self Study

1. What is Enum Data Type?

Enum is a value type in C# used to define a set of named constant values.

It is useful when a variable can have a **limited number of predefined values**.

Example:

```
enum OrderStatus
{
    Pending,
    Shipped,
    Delivered
}
```

Common uses:

- Days
- User Roles
- Order Status
- Directions

Three common built-in Enums:

1. DayOfWeek
 2. DateTimeKind
 3. StringComparison
-

2. String vs StringBuilder

Both are used to work with text, but they are suitable for different situations.

String:

- string is **immutable**, meaning its value cannot be changed after creation.
- Suitable when the text is simple or does not change frequently.
- Good for a small number of concatenation operations.

StringBuilder:

- Used when a string needs to be modified many times.
- More efficient for repeated operations such as `Append()`, especially inside loops.
- Helps reduce the creation of many temporary string objects.

Example:

```
StringBuilder sb = new StringBuilder();
```

```
sb.Append("Hello ");
sb.Append("Mohamed");
```

Rule:

Use `string` for normal text operations.

Use `StringBuilder` for frequent and repeated modifications.

3. What is a User Defined Constructor?

A **User Defined Constructor** is a constructor written by the programmer inside a class.

Its main role is to **initialize an object when it is created** and give its fields or properties their initial values.

Example:

```
class Student
{
    public string Name;
    public int Age;

    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

When we create the object:

```
Student s = new Student("Mohamed", 21);
```

The constructor automatically initializes `Name` and `Age`.

Main role:

- Initialize object data.
 - Set default or required values.
 - Make sure the object starts in a valid state.
-

4. Compare between Array and Linked List

Both `Array` and `Linked List` are data structures used to store collections of elements, but their internal structures are different.

Feature	Array	Linked List
Structure	Contiguous elements	Nodes connected by links
Size	Fixed in normal arrays	Dynamic
Access	Fast using index $O(1)$	Sequential access $O(n)$
Insertion	Can be expensive	Efficient when node position is known
Deletion	Can be expensive	Efficient when node position is known
Memory	Less overhead	More overhead for links

Array is preferred when:

We need fast access to elements using their index.

Linked List is preferred when:

We have frequent insertion and deletion operations and don't need fast random access.

In short:

Array → better for **fast access**.

Linked List → better for **frequent modifications**.